

TASK 3 A & B

1. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
rishi@RISHI-VP: /tmp
rishi@RISHI-VP /tmp$ ./task2.c
bash: ./task2.c: Permission denied
rishi@RISHI-VP /tmp$ gcc task2.c -o task2
rishi@RISHI-VP /tmp$ ./task2
Usage: ./task2 <source> <destination>
rishi@RISHI-VP /tmp$ echo "This is a test file for task 2" > source.txt
rishi@RISHI-VP /tmp$ ./task2 source.txt dest.txt
File copied successfully.
rishi@RISHI-VP /tmp$ cat dest.txt
This is a test file for task 2
rishi@RISHI-VP /tmp$ cd /tmp
rishi@RISHI-VP /tmp$ printf '#include <stdio.h>\n#include <unistd.h>\n#include <sys/wait.h>\n\nint main() {\n    pid_t pid = fork();\n\n    if (pid < 0) {\n        perror("fork failed");\n    } else if (pid == 0) {\n        printf("Child: PID=%d, PPID=%d\\n", getpid(), getppid());\n    } else {\n        printf("Parent: PID=%d, PPID=%d, Child PID=%d\\n", getpid(), getppid(), pid);\n        wait(NULL);\n        printf("Parent: child has terminated.\\n");\n    }\n    return 0;\n}' > task3.c
rishi@RISHI-VP /tmp$ cat task3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        printf("Child: PID=%d, PPID=%d\\n", getpid(), getppid());
    } else {
        printf("Parent: PID=%d, PPID=%d, Child PID=%d\\n", getpid(), getppid(), pid);
        wait(NULL);
        printf("Parent: child has terminated.\\n");
    }
    return 0;
}
rishi@RISHI-VP /tmp$ gcc task3.c -o task3
rishi@RISHI-VP /tmp$ ./task3
Parent: PID=1258, PPID=1173, Child PID=1259
Child: PID=1259, PPID=1258
```

OUTPUT :

```
rishi@RISHI-VP: /tmp
rishi@RISHI-VP /tmp$ cat dest.txt
This is a test file for task 2
rishi@RISHI-VP /tmp$ cd /tmp
rishi@RISHI-VP /tmp$ printf '#include <stdio.h>\n#include <unistd.h>\n#include <sys/wait.h>\n\nint main() {\n    pid_t pid = fork();\n\n    if (pid < 0) {\n        perror("fork failed");\n    } else if (pid == 0) {\n        printf("Child: PID=%d, PPID=%d\n", getpid(), getppid());\n    } else {\n        printf("Parent: PID=%d, PPID=%d, Child PID=%d\n", getpid(), getppid(), pid);\n        wait(NULL);\n        printf("Parent: child has terminated.\n");\n    }\n    return 0;\n}\n' > task3.c
rishi@RISHI-VP /tmp$ cat task3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        printf("Child: PID=%d, PPID=%d\n", getpid(), getppid());
    } else {
        printf("Parent: PID=%d, PPID=%d, Child PID=%d\n", getpid(), getppid(), pid);
        wait(NULL);
        printf("Parent: child has terminated.\n");
    }
    return 0;
}
rishi@RISHI-VP /tmp$ gcc task3.c -o task3
rishi@RISHI-VP /tmp$ ./task3
Parent: PID=1258, PPID=1173, Child PID=1259
Child: PID=1259, PPID=1258
Parent: child has terminated.
rishi@RISHI-VP /tmp$ ./task3
Parent: PID=1262, PPID=1173, Child PID=1263
Child: PID=1263, PPID=1262
Parent: child has terminated.
rishi@RISHI-VP /tmp$
```

3B:

```
rishi@RISHI-VP: /tmp
}
rishi@RISHI-VP /tmp$ gcc task3.c -o task3
rishi@RISHI-VP /tmp$ ./task3
Parent: PID=1258, PPID=1173, Child PID=1259
Child: PID=1259, PPID=1258
Parent: child has terminated.
rishi@RISHI-VP /tmp$ ./task3
Parent: PID=1262, PPID=1173, Child PID=1263
Child: PID=1263, PPID=1262
Parent: child has terminated.
rishi@RISHI-VP /tmp$ sleep 30 &
[1] 1296
rishi@RISHI-VP /tmp$ ps -o pid,ppid,stat,cmd -p 12345
  PID   PPID  STAT  CMD
rishi@RISHI-VP /tmp$ top
top - 04:06:20 up 1:03, 2 users, load average: 0.01, 0.05, 0.01
Tasks: 35 total, 1 running, 34 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7869.6 total, 7310.3 free, 511.7 used, 198.1 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7357.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
232	message+	20	0	9660	5456	4736	S	0.0	0.1	0:00.31	dbus-daemon
242	root	20	0	2221260	39612	25704	S	0.0	0.5	0:02.98	snappd
245	root	20	0	18148	8860	7796	S	0.0	0.1	0:00.30	systemd-logind
262	syslog	20	0	222516	5980	4576	S	0.0	0.1	0:00.22	rsyslogd
264	root	20	0	3124	1968	1832	S	0.0	0.0	0:00.02	agetty
297	root	20	0	107036	23144	13692	S	0.0	0.3	0:00.29	unattended-upgr
494	root	20	0	6676	4500	3728	S	0.0	0.1	0:00.02	login
544	root	20	0	20396	11628	9456	S	0.0	0.1	0:00.33	systemd
545	root	20	0	21172	3516	1804	S	0.0	0.0	0:00.00	(sd-pam)
567	root	20	0	6080	5332	3692	S	0.0	0.1	0:00.06	bash
1126	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	SessionLeader
1127	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.16	Relay(1128)
1128	root	20	0	6080	5348	3652	S	0.0	0.1	0:00.07	bash
1150	root	20	0	6808	4444	3980	S	0.0	0.1	0:00.00	su
1153	sairam	20	0	20404	11640	9464	S	0.0	0.1	0:00.25	systemd
1154	sairam	20	0	21164	3592	1868	S	0.0	0.0	0:00.00	(sd-pam)